



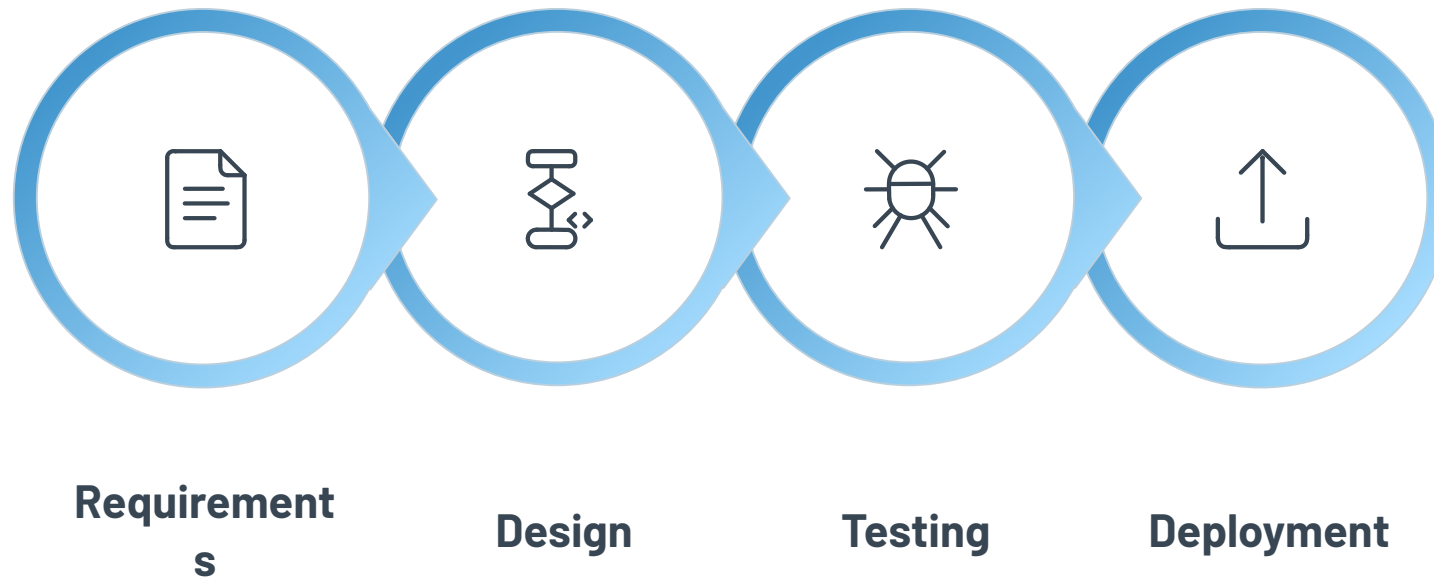
Agile Methodology in Software Development Life Cycle (SDLC)

An Overview of Agile Development Approach

Team: Raunak Shivhare · Brahm · Kartik · Neeraj Singh

What Is the Software Development Life Cycle?

The **Software Development Life Cycle (SDLC)** is a structured framework that guides development teams through the complete process of designing, building, testing, and deploying software. It defines clear phases, deliverables, and quality checkpoints to ensure predictable, high-quality outcomes.



Following a disciplined SDLC reduces project risk, improves resource allocation, and ensures that software meets both technical specifications and user expectations. Without a structured lifecycle, projects are prone to scope creep, missed deadlines, and quality failures.

CORE CONCEPT

Introduction to Agile Methodology

Agile is an iterative approach to software development that delivers work in small, consumable increments. Introduced through the **Agile Manifesto in 2001**, it was a response to the rigidity of traditional Waterfall models — emphasizing people, collaboration, and adaptability over rigid processes and documentation.

The Four Agile Values

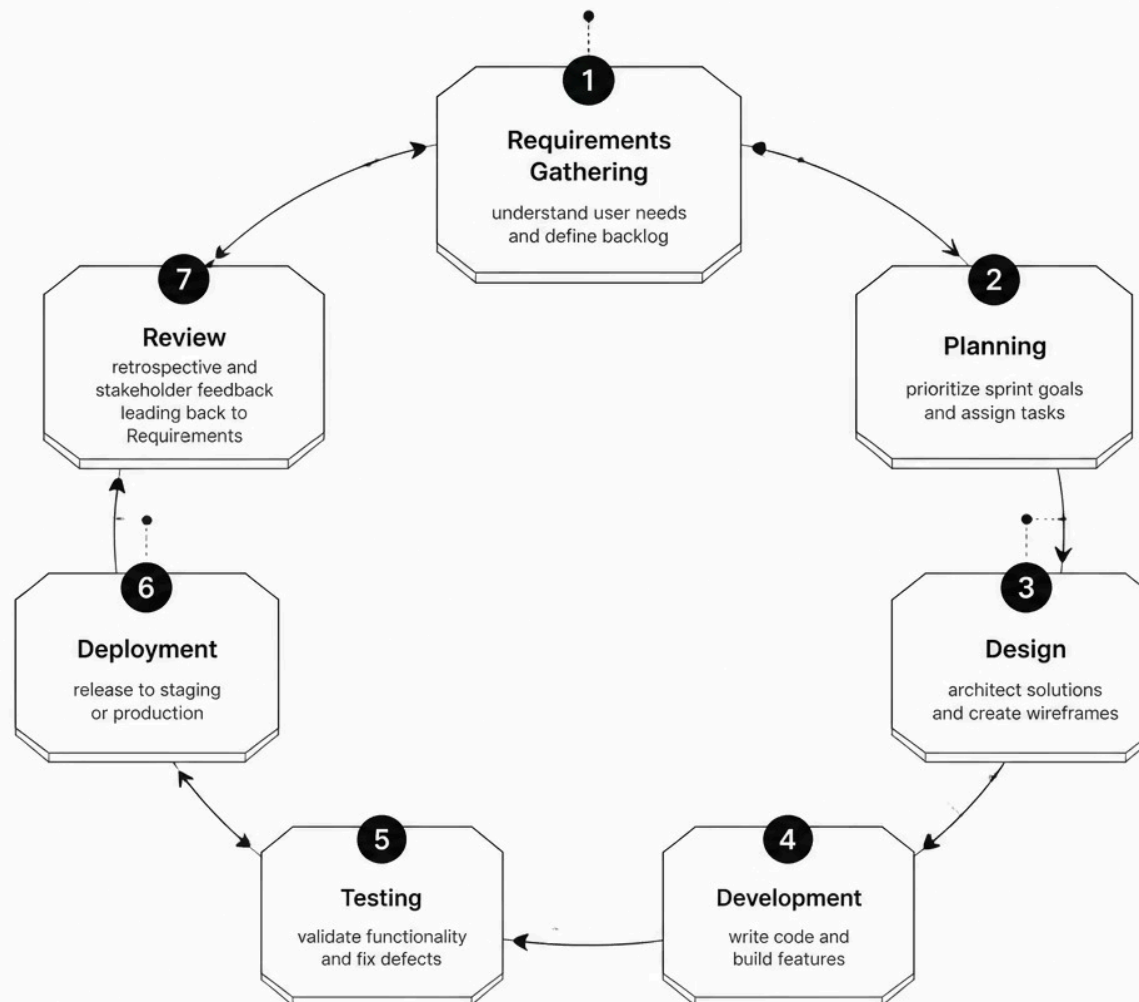
- Individuals & interactions over processes & tools
- Working software over comprehensive documentation
- Customer collaboration over contract negotiation
- Responding to change over following a plan

Why Agile Was Introduced

By the late 1990s, software projects frequently failed due to inflexible planning, poor communication, and an inability to respond to changing requirements. Agile emerged to address these systemic failures by placing **people, feedback, and adaptability** at the center of development.

The Agile Development Life Cycle

Unlike linear models, the Agile lifecycle is **circular and iterative**. Each sprint cycles through all phases, allowing the team to incorporate feedback and refine the product continuously. This loop ensures that the software evolves in alignment with real user needs and changing business priorities.



Each sprint typically lasts **1 to 4 weeks**, after which a potentially shippable product increment is delivered. The Review and Retrospective phases feed directly into the next cycle's planning, creating a self-improving development rhythm.

Key Features of Agile

Agile methodology is defined by a set of core characteristics that distinguish it from traditional development approaches. These features work together to create a responsive, team-driven, and customer-focused development environment.



Iterative Development

Work is broken into short sprints, each producing a functional increment that builds toward the final product.



Continuous Feedback

Regular reviews and retrospectives allow teams to adapt quickly and improve with each iteration.



Incremental Delivery

Working software is delivered frequently, providing early value and reducing the risk of large-scale failure.



Customer Collaboration

Stakeholders are engaged throughout the process, not just at the beginning and end, ensuring alignment with real needs.



Flexibility & Adaptability

Requirements can evolve mid-project without derailing the entire plan, making Agile ideal for dynamic environments.



Team Collaboration

Cross-functional teams work together daily, breaking down silos and fostering shared ownership of outcomes.

BENEFITS

Advantages of Agile Methodology

Organizations that adopt Agile consistently report significant improvements across delivery speed, product quality, and team morale. The framework's emphasis on iteration and collaboration creates a compounding advantage over time.



Faster Delivery

Short sprints and prioritized backlogs enable teams to release value to users much earlier than with Waterfall.



Improved Quality

Continuous testing and integration throughout each sprint catches defects early, reducing costly rework.



Customer Satisfaction

Frequent demos and feedback loops ensure the product evolves in line with stakeholder expectations.



Enhanced Collaboration

Daily stand-ups and shared ownership foster strong team dynamics and transparent communication.



Reduced Project Risks

Incremental delivery and early validation minimize the chance of catastrophic project failure.



Greater Flexibility

Changing requirements are welcomed, not resisted, allowing the product to stay relevant in dynamic markets.

Disadvantages of Agile Methodology

While Agile offers many advantages, it is not without challenges. Understanding its limitations helps teams implement it more effectively and set realistic expectations with stakeholders.

Requires Experienced Team Members

Agile relies on self-organizing, cross-functional teams. Without experienced practitioners, the lack of rigid structure can lead to confusion and inefficiency.

Difficult to Predict Timelines & Costs

Because requirements evolve throughout the project, it can be challenging to provide fixed delivery dates or budgets upfront — a concern for stakeholders who need certainty.

Frequent Changes Can Impact Scope

While adaptability is a strength, constant requirement changes can lead to scope creep if not managed carefully through a well-maintained product backlog.

Requires Continuous Customer Involvement

Agile demands active stakeholder participation throughout the project. If customers are unavailable or disengaged, the feedback loop breaks down.

Documentation May Be Limited

Agile prioritizes working software over comprehensive documentation, which can create challenges for knowledge transfer, compliance, and long-term maintenance.

Agile in Practice: Industries & Comparison

Agile has been adopted across industries — from technology and finance to healthcare and aerospace. Companies like **Spotify, Google, Amazon, and Microsoft** rely on Agile frameworks (Scrum, Kanban, SAFe) to deliver products at scale with speed and quality.

Agile vs. Waterfall: Key Differences

Dimension	Agile	Waterfall
Approach	Iterative & incremental	Sequential & linear
Requirements	Flexible, evolving	Fixed at the start
Customer Involvement	Continuous throughout	Mainly at start & end
Delivery	Frequent increments	Single final delivery
Testing	Integrated each sprint	Separate phase at end
Best For	Dynamic, uncertain projects	Well-defined, stable projects

❏ Agile is not a replacement for all methodologies — the right choice depends on project complexity, team maturity, and stakeholder requirements.

Conclusion & Key Takeaways

Agile methodology has fundamentally transformed how software is built — shifting the focus from rigid plans to **people, collaboration, and continuous improvement**. Its widespread adoption across industries is a testament to its effectiveness in delivering high-quality software in dynamic environments.

Iterative by Design

Small, frequent releases reduce risk and accelerate value delivery.

People-First

Collaboration and communication drive better outcomes than process alone.

Adapt to Change

Embracing change — rather than resisting it — is Agile's greatest strength.

Thank You

We appreciate your time and attention. This presentation was prepared by **Raunak Shivhare, Brahm, Kartik, and Neeraj** as part of the Software Engineering course curriculum.

❓ **Questions & Discussion** — We welcome your questions, comments, and insights on Agile methodology and its applications in modern software development.